

## Performance Testing

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| <b>Date</b>          | 2 JULY 2026                                                  |
| <b>Team ID</b>       | xxxxxx                                                       |
| <b>Project Name</b>  | OptiCrop – Smart Agricultural Production Optimization Engine |
| <b>Maximum Marks</b> | 5 Marks                                                      |

### Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

#### Step 1: Testing Overview

| Field                      | Details                                               |
|----------------------------|-------------------------------------------------------|
| <b>Testing Tool Used</b>   | Postman, Browser (Google Chrome)                      |
| <b>Type of Testing</b>     | Functional Testing & Performance Testing              |
| <b>Target Module / API</b> | Flask /predict API                                    |
| <b>Test Environment</b>    | Local System (Windows 10, VS Code, Python 3.9, Flask) |
| <b>Test Date</b>           | 5 JULY 2026                                           |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description  | No. of Virtual Users | Duration (sec) | Expected Outcome                                 |
|------|------------------------------|----------------------|----------------|--------------------------------------------------|
| 1    | Single Crop Prediction       | 1                    | 10             | Crop prediction should be displayed successfully |
| 2    | Multiple Prediction Requests | 5                    | 30             | All predictions should be processed correctly    |
| 3    | Invalid Input Testing        | 1                    | 15             | System should display validation error           |

|   |                                |    |    |                                            |
|---|--------------------------------|----|----|--------------------------------------------|
| 4 | Continuous Prediction Requests | 10 | 60 | Flask application should remain responsive |
|---|--------------------------------|----|----|--------------------------------------------|



### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value | Status (Pass / Fail) | Remarks                      |
|------|----------------------|--------------|--------------|----------------------|------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 0.48 sec     | Pass                 | Fast prediction              |
| 2    | Response Time (Max)  | < 5 seconds  | 1.10 sec     | Pass                 | Within acceptable limit      |
| 3    | Throughput (Req/sec) | 5+           | 12 req/sec   | Pass                 | Good performance             |
| 4    | Error Rate           | < 1%         | 0%           | Pass                 | No errors observed           |
| 5    | CPU Utilization      | < 80%        | 28%          | Pass                 | Low CPU usage                |
| 6    | Memory Utilization   | < 80%        | 34%          | Pass                 | Efficient memory utilization |

### Step 4: Observations & Analysis

- The application successfully handled all prediction requests.
- The Flask backend responded quickly with low latency.
- No crashes or performance degradation were observed.
- Input validation prevented invalid data from being processed.
- CPU and memory usage remained within acceptable limits.
- The Random Forest model generated predictions accurately and efficiently.

### Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

